Adversarial search and Minimax Algorithm



Zola Mahlaza

Department of Computer Science

Feb 2026

Topics

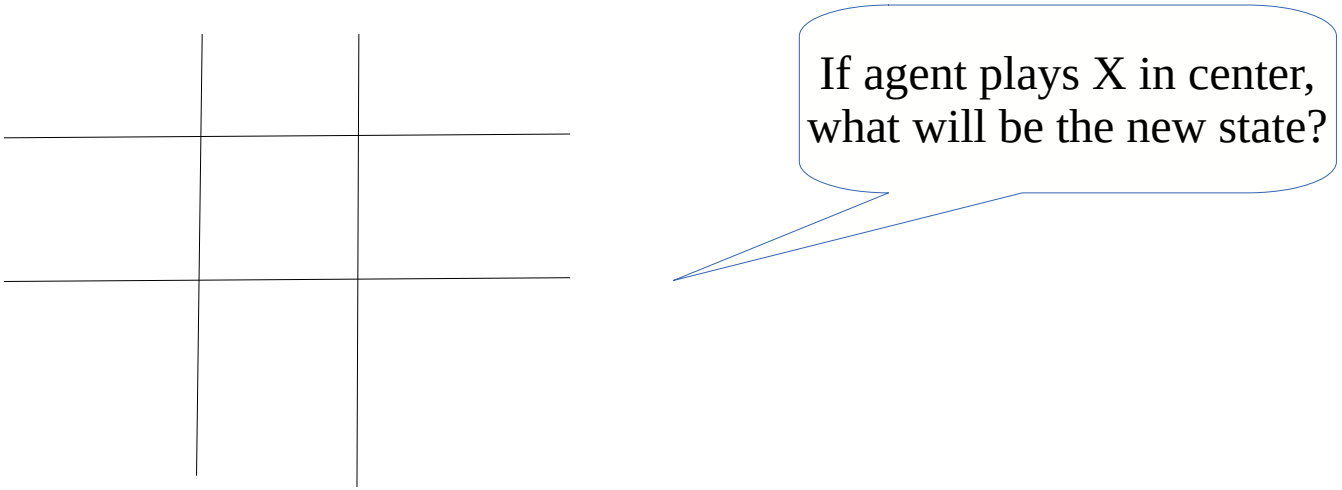
- Adversarial search and games
- The Minimax Algorithm

Introduction

- Framework for problem solving, so far
  - Problem instance → [search algo.] → optimal solution
- Techniques work well for pathfinding
- Competitive environments: two or more agents have conflicting goals
  - Presence of multiple agents
  - Conflicting utility
- One approach we can take: aggregate as an economy
  - Zoom-out and focus on patterns (e.g., increasing demand will cause prices to rise)
  - Market myopia: agents may make decision based on current 'prices'

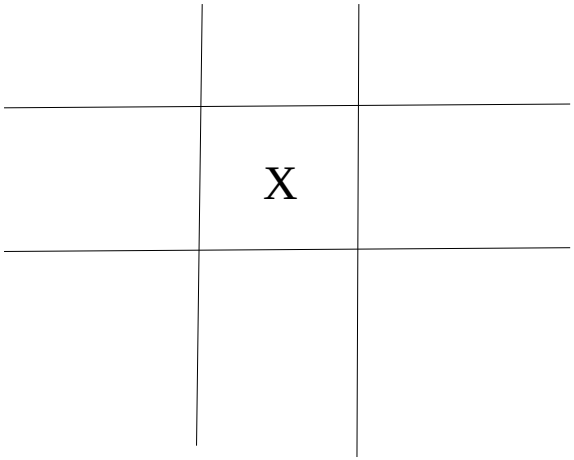
Adversarial search

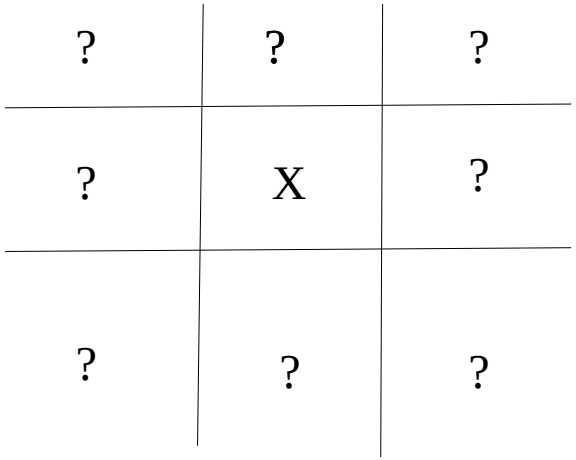
- Second approach: adversaries are part of the environment
  - Environment then becomes non-deterministic
  - Agent no longer knows what state it transitions to after taking an action



Adversarial search

- Second approach: adversaries are part of the environment
  - Environment then becomes non-deterministic
  - Agent no longer knows what state it transitions to after taking an action

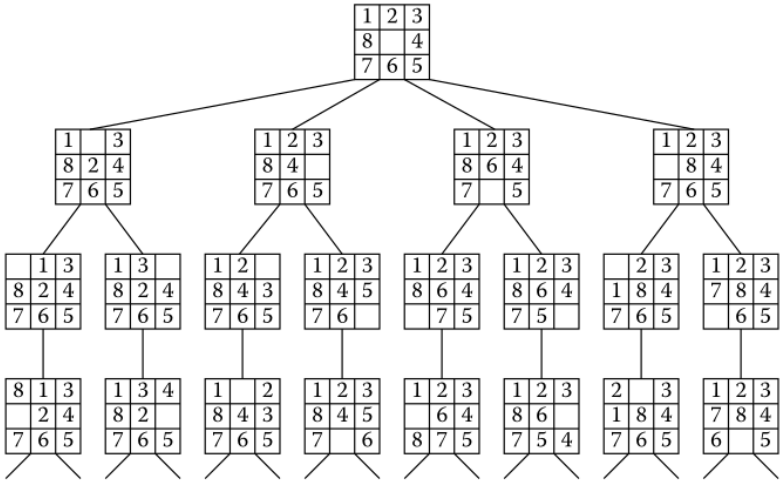




Adversarial search

- Challenge: we are not modeling the intention of the adversary

- Third approach: explicit model of agents + game-tree search



Include agents

Game properties

- We focus on games have the properties:
  - Deterministic:
  - Two-player
  - Turn-taking
  - Perfect information: both players see the entire state at all times
  - Zero-sum: What’s good for one player is bad for the other

Adversarial search: agents

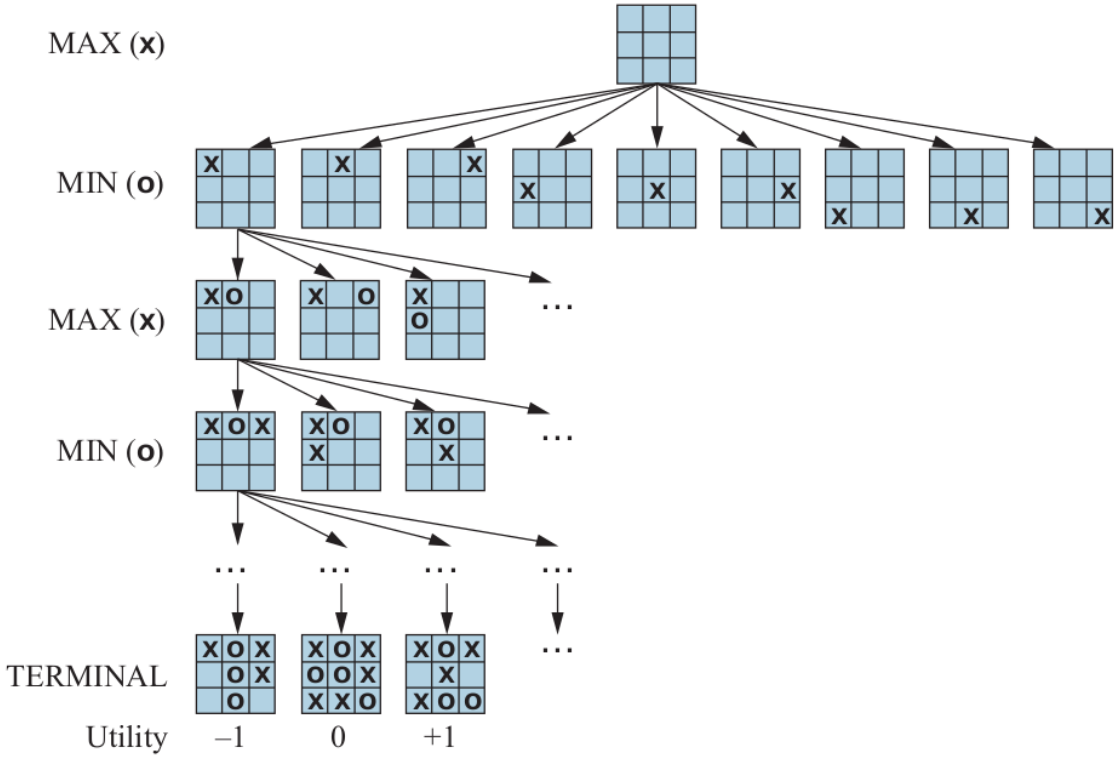
- In case of two agents that are adversaries
  - Agent that moves first is called MAX. The other one is the MIN.
- At the end of the game, winner gets points & loser is penalised
- We need game-specific definitions

Toward game tree

- We focus on games have the properties:
  - Initial state: how the game starts
  - ToMove(s): whose turn is it to move?
  - Result(s,a): transition model
  - Actions(s): Available actions at state
  - Is-terminal(s): Method for determining whether we are in terminal state
  - Utility(s, p): payoff function. Assigns numerical score to player in terminal state (e.g, draw=0.5, win=1, lose = 0)
- Use Initial state, actions, and state transition to define state space graph

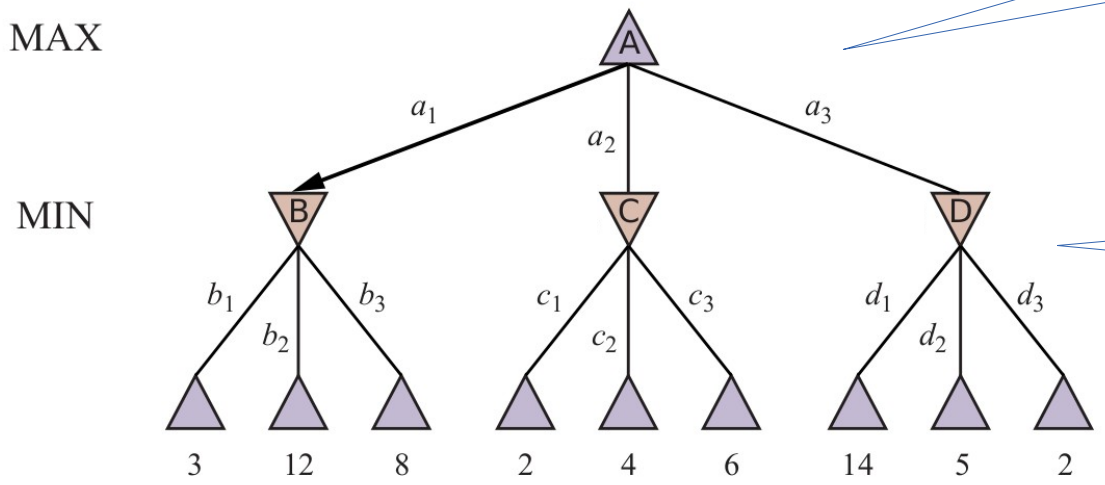
Game tree

- Complete game tree: search tree. Follows every move until a terminal node reached
- Game tree may be infinite
  - (when state space is infinite)



Game tree

- Solving tic-tac-toe is not difficult + not demanding
  - Number of leaf nodes = 9!
- You cannot generate all leaf nodes for chess
  - Min. number of leaf nodes 10^{40}
- Consider the game:



What is the best action for Max?

What is the best action for Min?

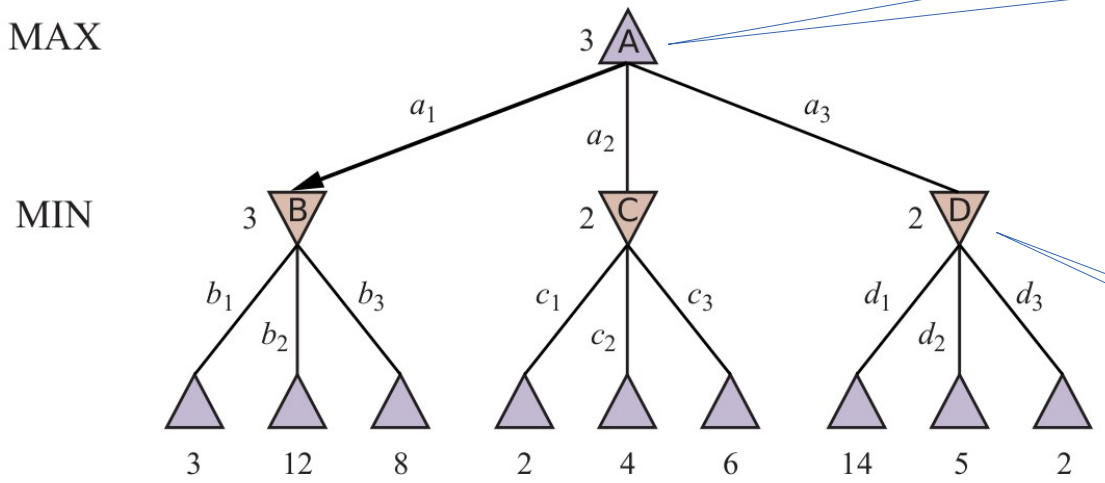
Minimax value

- At each node, "best" outcome a player can achieve modeled using minimax value
  - Utility of state if both agents are playing optimally
- Minimax value at terminal = its utility

$$minimax(s) = \begin{cases} utility(s, MAX) & terminal(s) \\ \max_{a \in actions} minimax(result(s,a)) & tomove(s) = MAX \\ \min_{a \in actions} minimax(result(s,a)) & tomove(s) = MIN \end{cases}$$

Minimax value

- Minimax value calculation:



Max{3, 2, 2}

Min{14, 5, 2}

Minimax algorithm

- If opponent does not play first:

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move ← v2, a
    return v, move
```

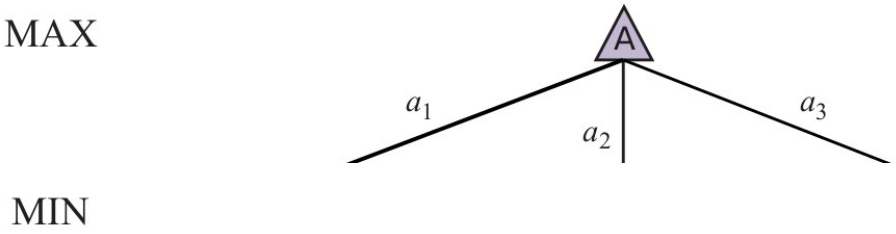
Minimax example

- Minimax value calculation steps:



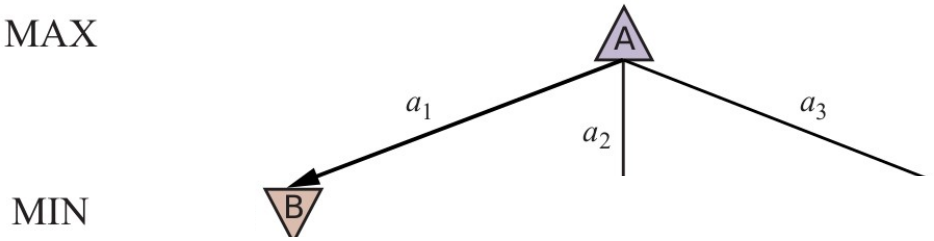
Minimax example

- Minimax value calculation steps:



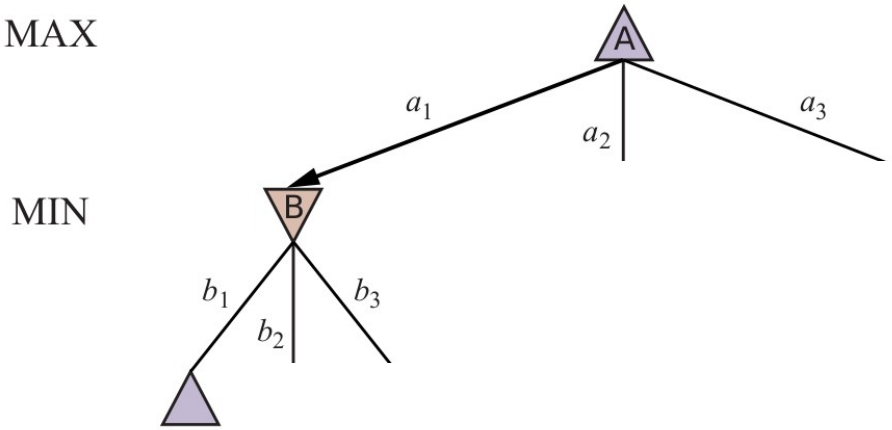
Minimax example

- Minimax value calculation steps:



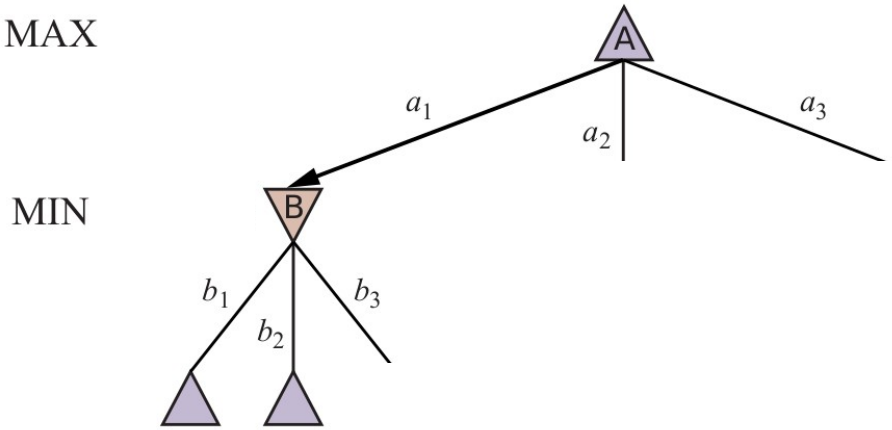
Minimax example

- Minimax value calculation steps:



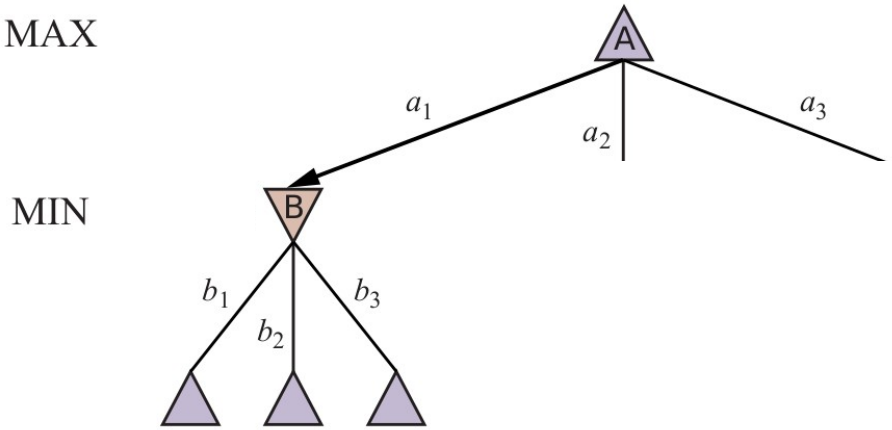
Minimax example

- Minimax value calculation steps:



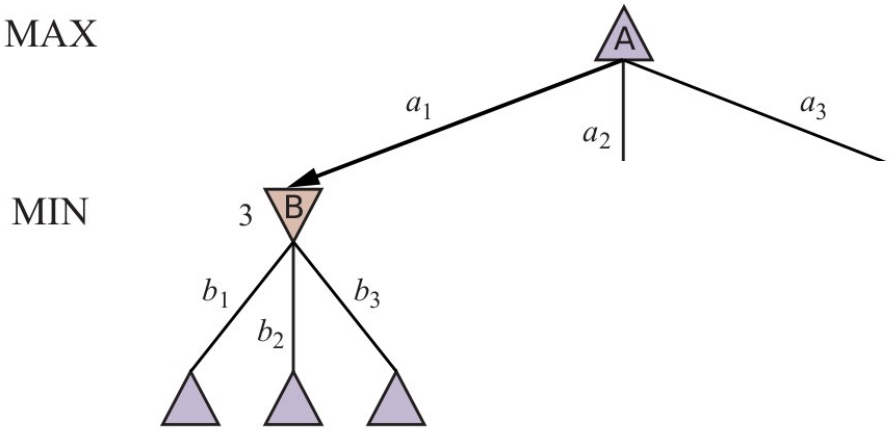
Minimax example

- Minimax value calculation steps:



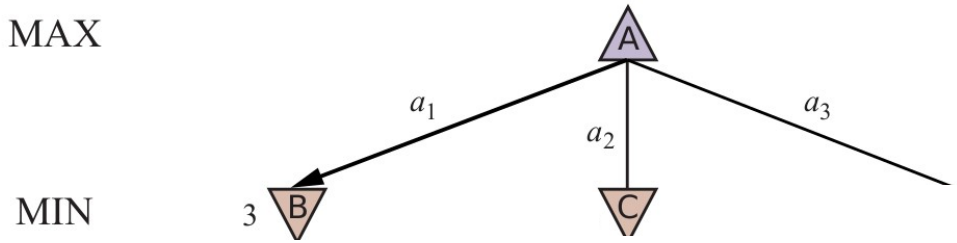
Minimax example

- Minimax value calculation steps:



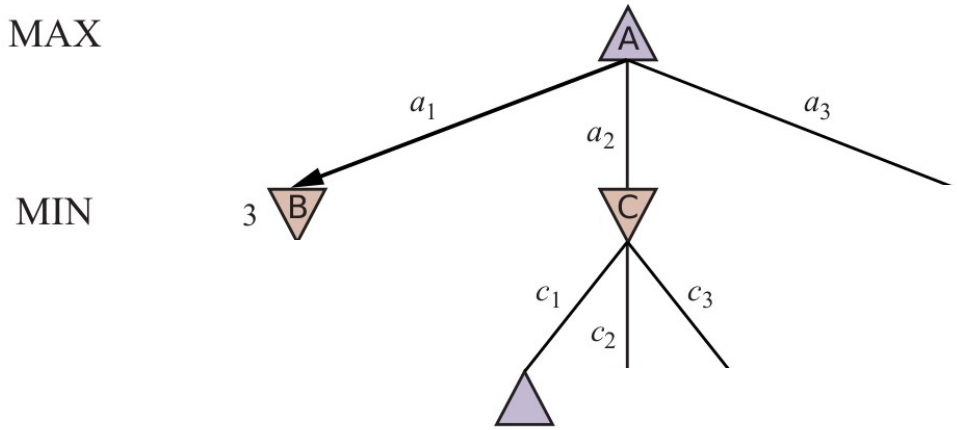
Minimax example

- Minimax value calculation steps:



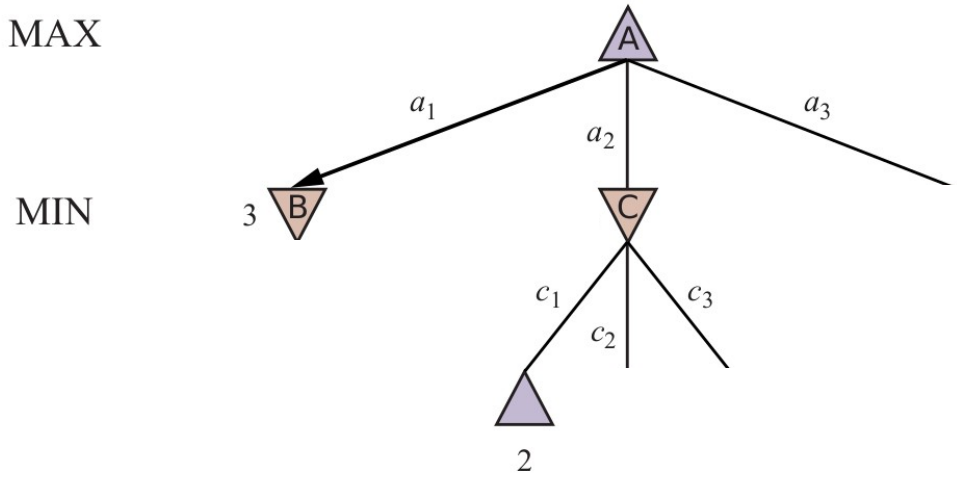
Minimax example

- Minimax value calculation steps:



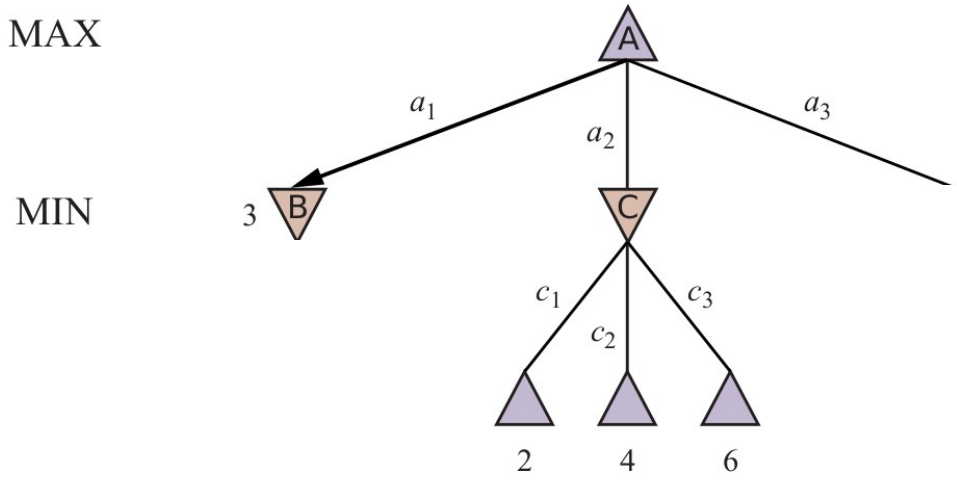
Minimax example

- Minimax value calculation steps:



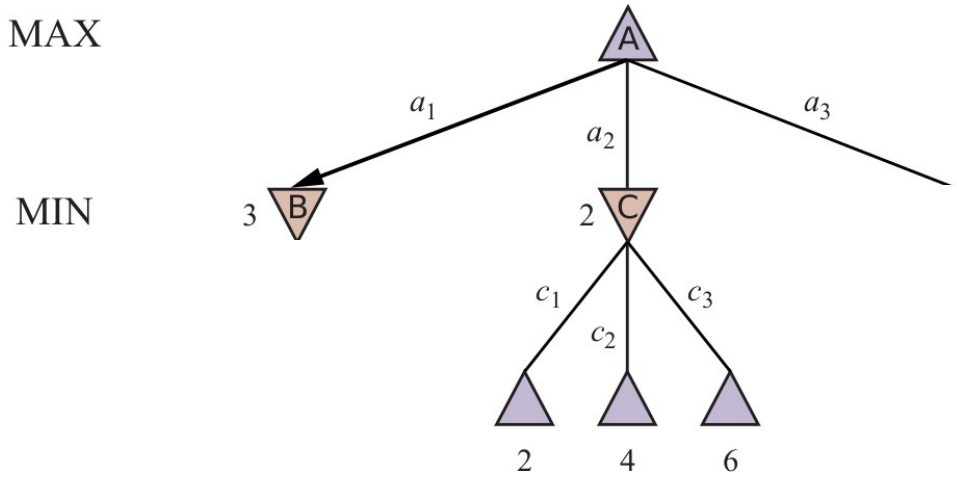
Minimax example

- Minimax value calculation steps:



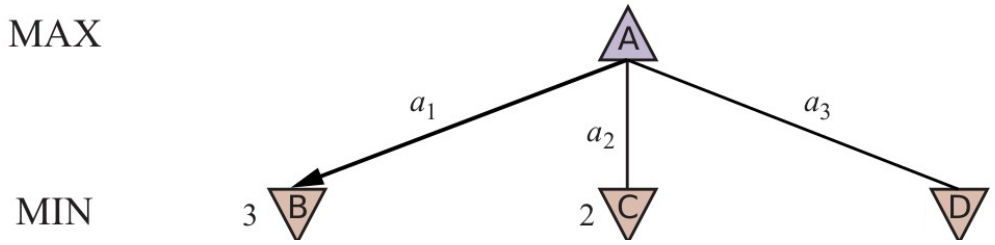
Minimax example

- Minimax value calculation steps:



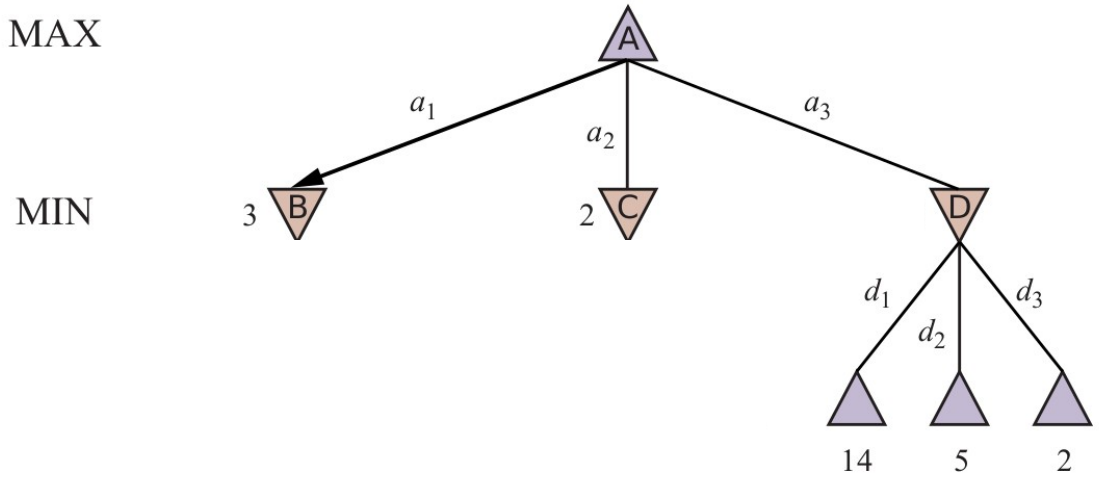
Minimax example

- Minimax value calculation steps:



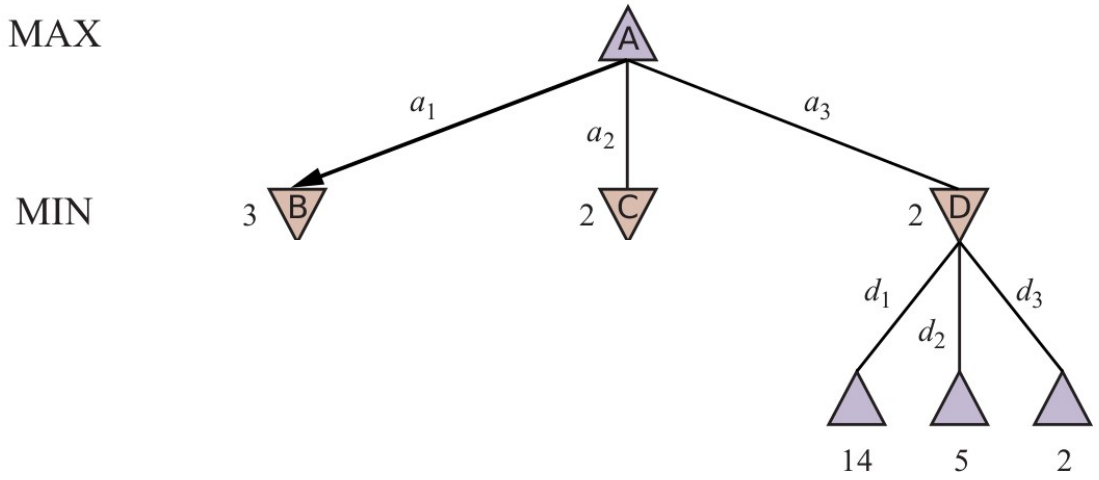
Minimax example

- Minimax value calculation steps:



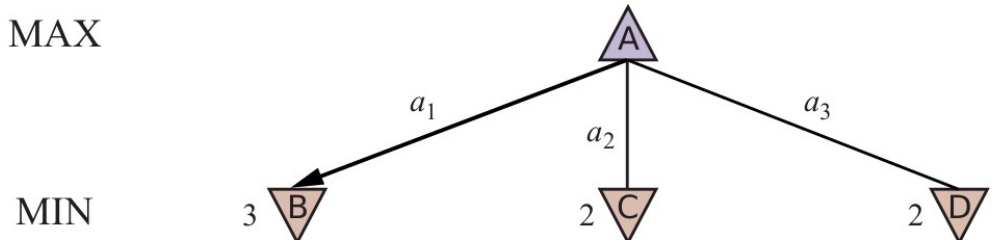
Minimax example

- Minimax value calculation steps:



Minimax example

- Minimax value calculation steps:



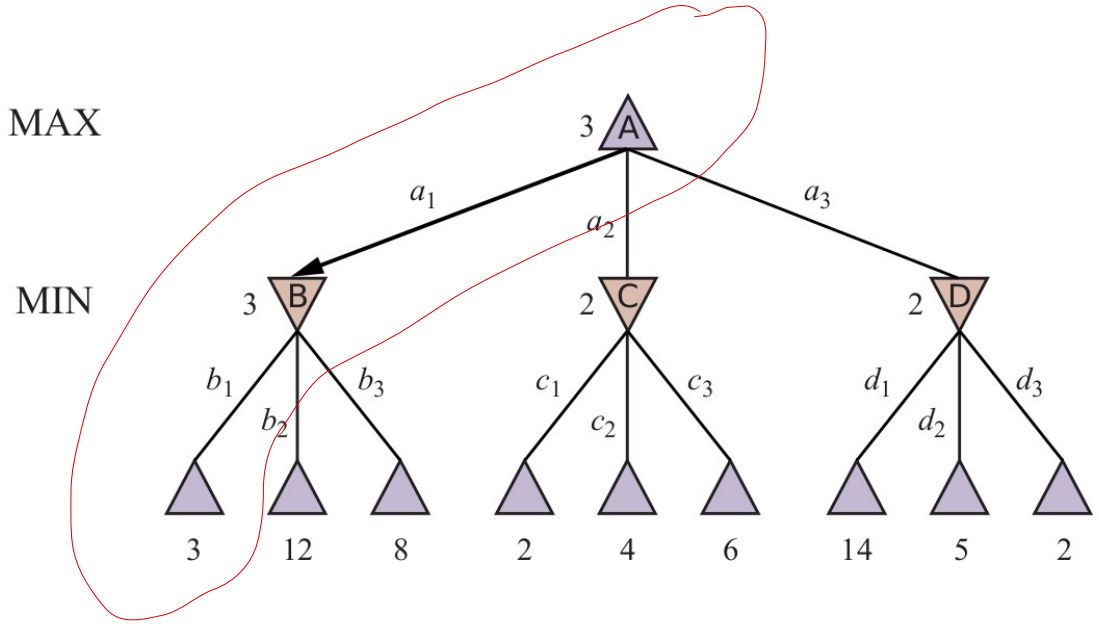
Minimax example

- Minimax value calculation steps:



Max action, min action

- Actions to be taken by each agent



- This definition assumes that each agents acts optimally

Minimax algorithm

- It may be sensible to consider that adversary may sub-optimally
- Example:
  - Max is optimal, but Min is not
  - Case 1: if max continues to play optimally, game will end in a draw
  - Case 2: max can play risky move and 9/10 actions available to min will lead to max winning (i.e., min may blunder).
  - What should you do?

Minimax algorithm

- Minimax completes depth-first exploration of game tree
  - Path that matters is the one with
- Time complexity is $O(b^m)$ where b = branching factor, m = max depth
- Space complexity is O(bm).
  - Once the minimax value is computed for upper level, dispose of the nodes

Minimax algorithm

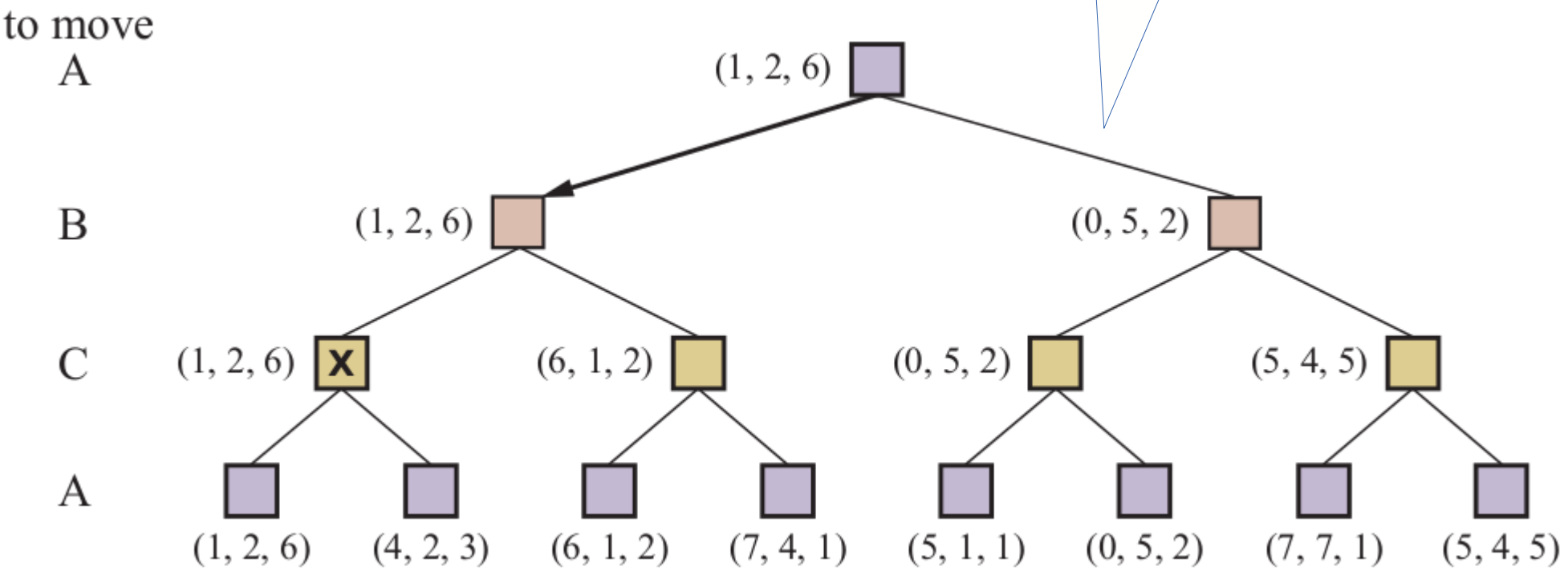
- Exponential time complexity is a limiting factor
- Consider chess:
  - Branching factor approx. 35, avg depth of 80
  - Searching through 35^{80} states = Three quadragintillion …
- Algorithm useful for analysing games

Multiplayer games

- A number of games are multiplayer
- Extending minimax:
  - single value in node becomes a vector
- In leaf nodes, utility of the state is given from each player's viewpoint
  - UTILITY function can return a vector

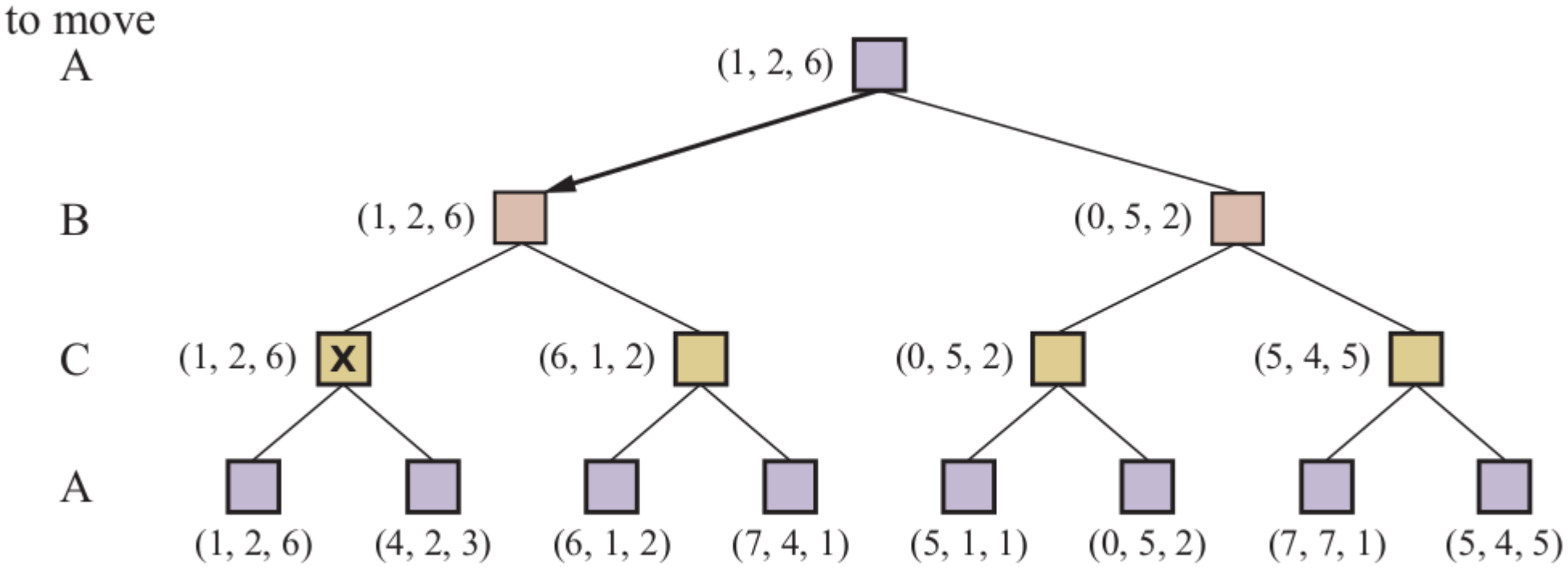
Multiplayer games

How to calculate value of Non-terminal states?



Multiplayer games

- Value of n is based on successor state:
  - Choose highest value for the player choosing at n
- Assume third pos. in vector is for agent C:



Multiplayer games

- Competition: Usage of max assumes each agent is purely selfish
- Cooperation:
  - Treat agent B and C as working against A
  - Turn problem to zero-sum game where (B+C) are a single ‘agent’. Sum their scores.
  - If hardcoded, method does not account for shifting loyalties
- Shifting alliances
  - When calculating utility of C, use a weighted (W_A and W_B) combination of the other agents
  - Weights change based on game state

Reading

- Sections 5.1 - 5.2

